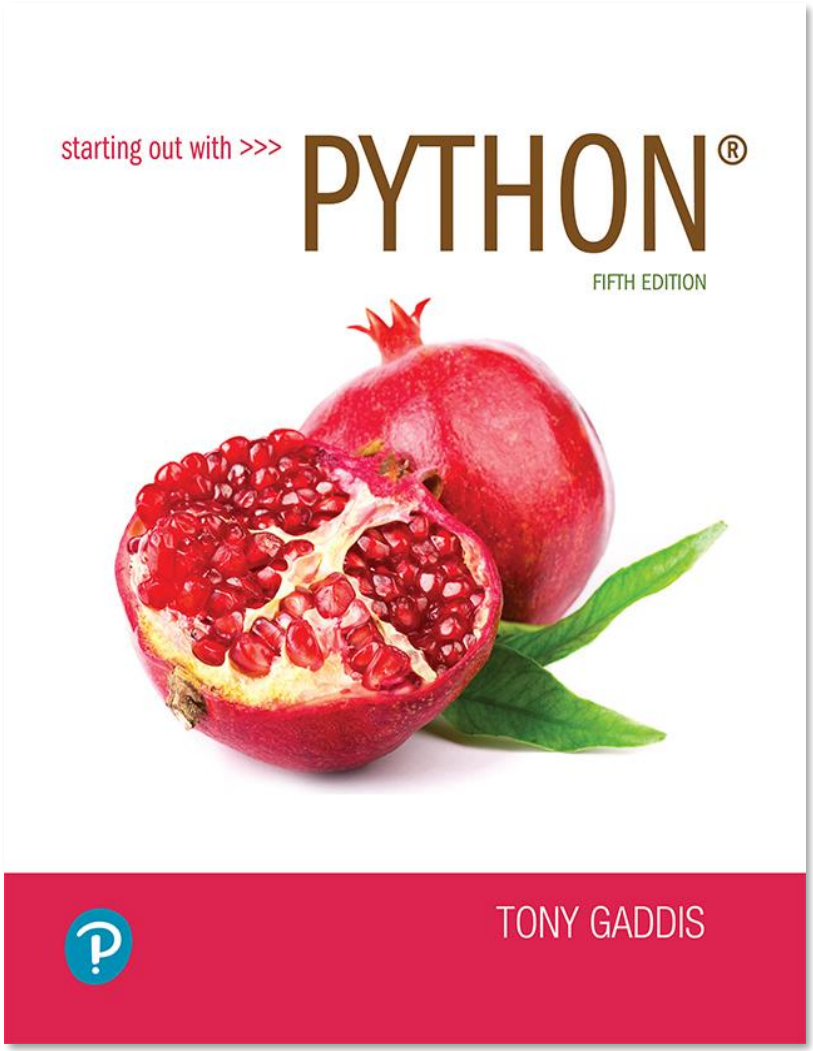
# Starting out with Python

Fifth Edition




## Chapter 8

More About Strings

# Topics

- Basic String Operations
- String Slicing
- Testing, Searching, and Manipulating Strings

# Basic String Operations

- Many types of programs perform operations on strings (word processing, emails, search engines, etc.)
- In Python, many tools for examining and manipulating strings
  - Strings are sequences, so many of the tools that work with sequences work with strings

# Accessing the Individual Characters in a String (1 of 4)

- **To access an individual character in a string:**
  - Use a **`for` loop**
    - **Format: `for character in string:`**
    - Useful when need to iterate over the whole string, such as to count the occurrences of a specific character
  - Use indexing
    - Each character has an index specifying its position in the string, starting at 0
    - **Format: `character = my_string[i]`**

# Accessing the Individual Characters in a String (2 of 4)

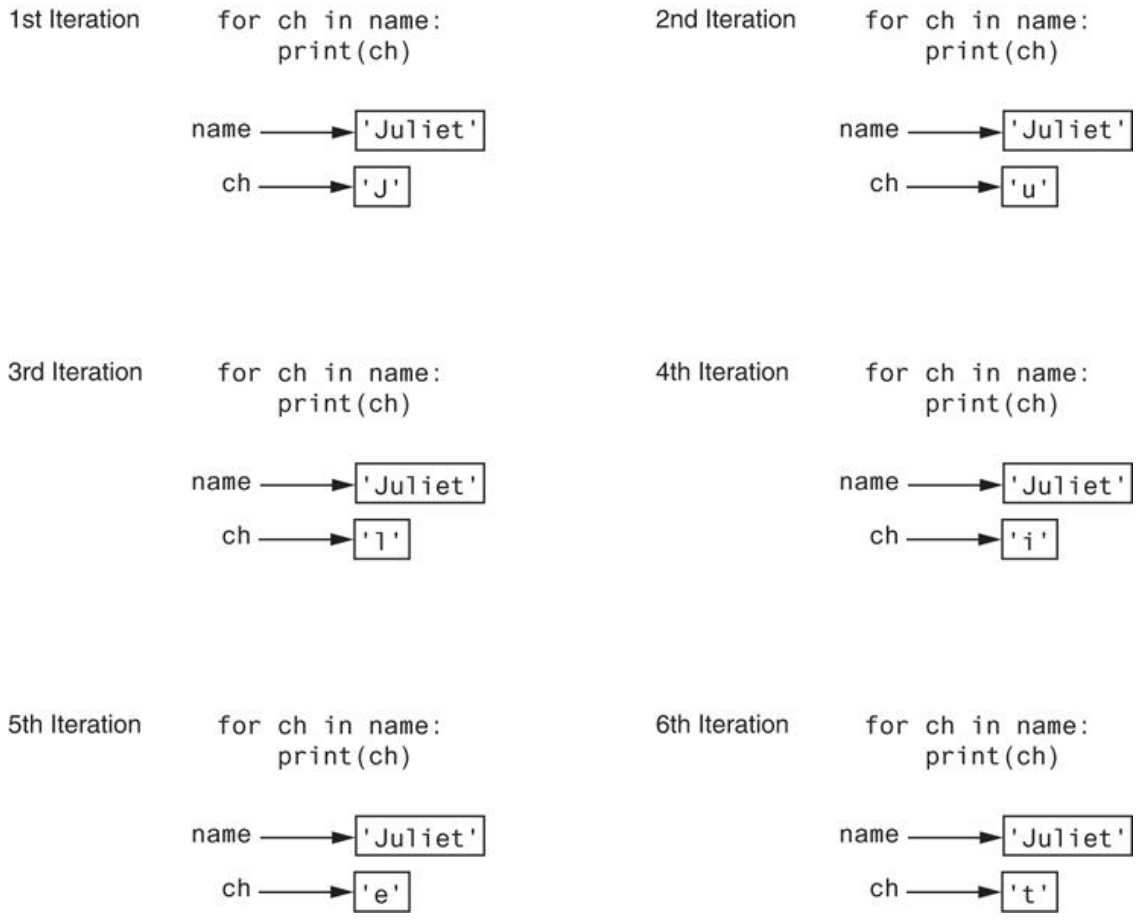



**Figure 8-1** Iterating over the string 'Juliet'

# Accessing the Individual Characters in a String (3 of 4)

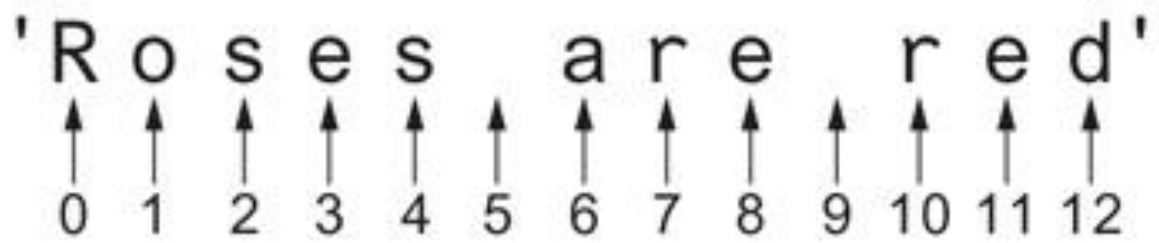



**Figure 8-2** String indexes

Can use positive / Negative Index

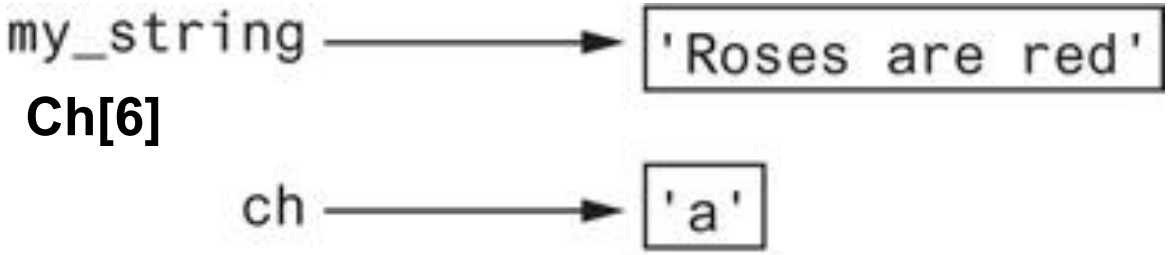



**Figure 8-3** Getting a copy of a character from a string

```
def main():
    # Create a variable to use to hold the count.
    # The variable must start with 0.
    count = 0

    # Get a string from the user.
    my_string = input('Enter a sentence: ')

    # Count the Ts.
    for ch in my_string:
        if ch == 'T' or ch == 't':
            count += 1

    # Print the result.
    print('The letter T appears', count, 'times.')
```

# Accessing the Individual Characters in a String (4 of 4)

- **`IndexError` exception will occur if:**
  - You try to use an index that is out of range for the string
    - Likely to happen when loop iterates beyond the end of the string
- **`len(string)`** function can be used to obtain the length of a string
  - Useful to prevent loops from iterating beyond the end of a string

# Accessing the Individual Characters in a String (cont'd.)

```
city = 'Boston'
index = 0
while index < 7:
    print(city[index])
    index += 1
```

```
city = 'Boston'
index = 0
while index < len(city):
    print(city[index])
    index += 1
```

# String Concatenation

- **Concatenation:** appending one string to the end of another string
  - Use the + operator to produce a string that is a combination of its operands
  - The augmented assignment operator += can also be used to concatenate strings
    - The operand on the left side of the += operator must be an existing variable; otherwise, an exception is raised

```
>>> message = 'Hello ' + 'world'
>>> print(message) Enter
Hello world
```

# Strings Are Immutable (1 of 2)

- Strings are **immutable**
  - Once they are created, they cannot be changed
    - Concatenation doesn't actually change the existing string, but rather creates a new string and assigns the new string to the previously used variable
  - Cannot use an expression of the form
  - ***`string[index] = new_character`***
    - Statement of this type will raise an exception

# Strings Are Immutable (2 of 2)

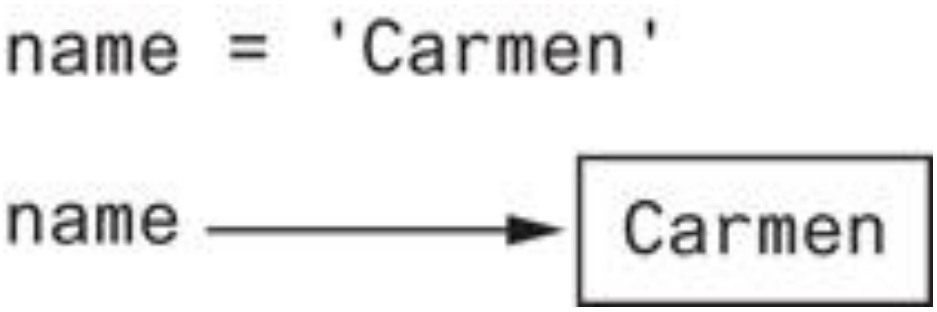



**Figure 8-4** The string 'Carmen' assigned to name

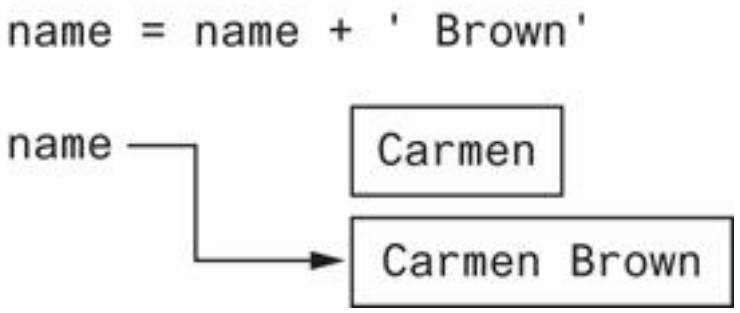



**Figure 8-5** The string 'Carmen Brown' assigned to name

# Test your understanding

**Checkpoint**

8.1 Assume the variable `name` references a string. Write a `for` loop that prints each character in the string.

8.2 What is the index of the first character in a string?

8.3 If a string has 10 characters, what is the index of the last character?

8.4 What happens if you try to use an invalid index to access a character in a string?

8.5 How do you find the length of a string?

8.6 What is wrong with the following code?

```
animal = 'Tiger'
animal[0] = 'L'
```

# String Slicing

- **Slice:** span of items taken from a sequence, known as *substring*
  - **Slicing format:** ***`string[start : end]`***
    - Expression will return a string containing a copy of the characters from *`start`* up to, but not including, *`end`*
    - If *`start`* not specified, `0` is used for start index
    - If *`end`* not specified, `len(string)` is used for end index
  - Slicing expressions can include a step value and negative indexes relative to end of string

# Extracting Characters from a String

```
def get_login_name(first, last, idnumber):
    # Get the first three letters of the first name.
    # If the name is less than 3 characters, the
    # slice will return the entire first name.
    set1 = first[0 : 3]

    # Get the first three letters of the last name.
    # If the name is less than 3 characters, the
    # slice will return the entire last name.
    set2 = last[0 : 3]

    # Get the last three characters of the student ID.
    # If the ID number is less than 3 characters, the
    # slice will return the entire ID number.
    set3 = idnumber[-3 :]

    # Put the sets of characters together.
    login_name = set1 + set2 + set3

    # Return the login name.
```

# Test your understanding

## Checkpoint

8.7 What will the following code display?

```
mystring = 'abcdefg'
print(mystring[2:5])
```

8.8 What will the following code display?

```
mystring = 'abcdefg'
print(mystring[3:])
```

8.9 What will the following code display?

```
mystring = 'abcdefg'
print(mystring[:3])
```

8.10 What will the following code display?

```
mystring = 'abcdefg'
print(mystring[:])
```

# Testing, Searching, and Manipulating Strings

- You can use the **`in`** operator to determine whether one string is contained in another string
  - **General format: *`string1 in string2`***
    - *`string1`* and *`string2`* can be string literals or variables referencing strings
- Similarly you can use the **`not in`** operator to determine whether one string is not contained in another string

# String Methods (1 of 7)

- Strings in Python have many types of methods, divided into different types of operations
  - **General format:**
    ***mystring.method(arguments)***
- Some methods test a string for specific characteristics
  - Generally Boolean methods, that return `True` if a condition exists, and `False` otherwise

# String Methods (2 of 7)

**Table 8-1** Some string testing methods

| Method | Description |
|---|---|
| `isalnum()` | Returns true if the string contains only alphabetic letters or digits and is at least one character in length. Returns false otherwise. |
| `isalpha()` | Returns true if the string contains only alphabetic letters and is at least one character in length. Returns false otherwise. |
| `isdigit()` | Returns true if the string contains only numeric digits and is at least one character in length. Returns false otherwise. |
| `islower()` | Returns true if all of the alphabetic letters in the string are lowercase, and the string contains at least one alphabetic letter. Returns false otherwise. |
| `isspace()` | Returns true if the string contains only whitespace characters and is at least one character in length. Returns false otherwise. (Whitespace characters are spaces, newlines (`\n`), and tabs (`\t`). |
| `isupper()` | Returns true if all of the alphabetic letters in the string are uppercase, and the string contains at least one alphabetic letter. Returns false otherwise. |

```
def main():
    # Get a string from the user.
    user_string = input('Enter a string: ')

    print('This is what I found about that string:')

    # Test the string.
    if user_string.isalnum():
       print('The string is alphanumeric.')
    if user_string.isdigit():
       print('The string contains only digits.')
    if user_string.isalpha():
       print('The string contains only alphabetic characters.')
    if user_string.isspace():
       print('The string contains only whitespace characters.')
    if user_string.islower():
       print('The letters in the string are all lowercase.')
    if user_string.isupper():
       print('The letters in the string are all uppercase.')
```

# String Methods (3 of 7)

- Some methods return a copy of the string, to which modifications have been made
  - Simulate strings as mutable objects
- String comparisons are case-sensitive
  - Uppercase characters are distinguished from lowercase characters
  - `lower` and `upper` methods can be used for making case-insensitive string comparisons

# String Methods (4 of 7)

**Table 8-2** String Modification Methods

| Method | Description |
|---|---|
| `lower()` | Returns a copy of the string with all alphabetic letters converted to lowercase. Any character that is already lowercase, or is not an alphabetic letter, is unchanged. |
| `lstrip()` | Returns a copy of the string with all leading whitespace characters removed. Leading whitespace characters are spaces, newlines (`\n`), and tabs (`\t`) that appear at the beginning of the string. |
| `lstrip(`*`char`*`)` | The *`char`* argument is a string containing a character. Returns a copy of the string with all instances of *`char`* that appear at the beginning of the string removed. |
| `rstrip()` | Returns a copy of the string with all trailing whitespace characters removed. Trailing whitespace characters are spaces, newlines (`\n`), and tabs (`\t`) that appear at the end of the string. |
| `rstrip(`*`char`*`)` | The *`char`* argument is a string containing a character. The method returns a copy of the string with all instances of *`char`* that appear at the end of the string removed. |
| `strip()` | Returns a copy of the string with all leading and trailing whitespace characters removed. |
| `strip(`*`char`*`)` | Returns a copy of the string with all instances of *`char`* that appear at the beginning and the end of the string removed. |
| `upper()` | Returns a copy of the string with all alphabetic letters converted to uppercase. Any character that is already uppercase, or is not an alphabetic letter, is unchanged. |

# String Methods (5 of 7)

- Programs commonly need to search for substrings
- Several methods to accomplish this:
  - **`endswith(`*`substring`*`)`**: checks if the string ends with *`substring`*
    - Returns `True` or `False`
  - **`startswith(`*`substring`*`)`**: checks if the string starts with *`substring`*
    - Returns `True` or `False`

# String Methods (6 of 7)

- Several methods to accomplish this (cont'd):
  - **`find(substring)`**: searches for *`substring`* within the string
    - Returns lowest index of the substring, or if the substring is not contained in the string, returns -1
  - **`replace(substring, new_string)`**:
    - Returns a copy of the string where every occurrence of *`substring`* is replaced with *`new_string`*

# String Methods (7 of 7)

**Table 8-3** Search and replace methods

| Method | Description |
|---|---|
| `endswith(substring)` | The *substring* argument is a string. The method returns true if the string ends with *substring*. |
| `find(substring)` | The *substring* argument is a string. The method returns the lowest index in the string where *substring* is found. If *substring* is not found, the method returns −1. |
| `replace(old, new)` | The *old* and *new* arguments are both strings. The method returns a copy of the string with all instances of *old* replaced by *new*. |
| `startswith(substring)` | The *substring* argument is a string. The method returns true if the string starts with *substring*. |

# Spotlight: Password Validator

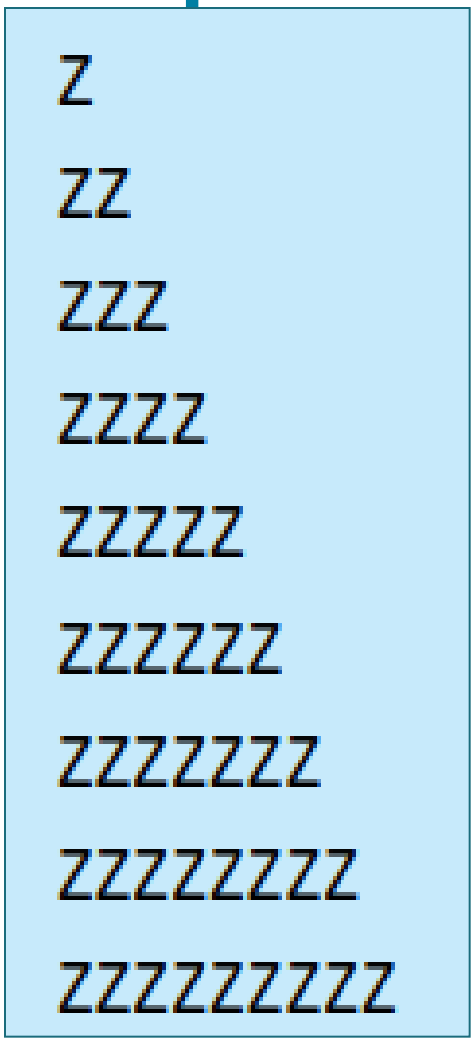
At the university, passwords for the campus computer system must meet the following requirements:

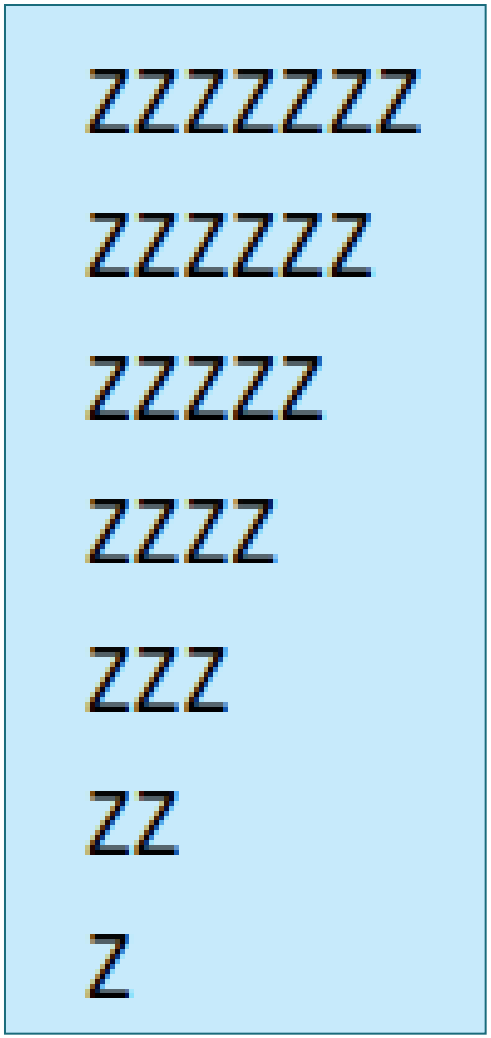
- The password must be at least seven characters long.
- It must contain at least one uppercase letter.
- It must contain at least one lowercase letter.
- It must contain at least one numeric digit.

# The Repetition Operator

- **Repetition operator:** makes multiple copies of a string and joins them together
  - The * symbol is a repetition operator when applied to a string and an integer
    - String is left operand; number is right
  - **General format:** ***string_to_copy * n***
  - Variable references a new string which contains multiple copies of the original string

# The Repetition Operator



# Splitting a String (1 of 2)

- `split` method: returns a list containing the words in the string
  - By default, uses space as separator
  - Can specify a different separator by passing it as an argument to the `split` method

# Splitting a String (2 of 2)

- Examples:

```
>>> my_string = 'One two three four'
>>> word_list = my_string.split()
>>> word_list
['One', 'two', 'three', 'four']
>>>
```

```
>>> my_string = '1/2/3/4/5'
>>> number_list = my_string.split('/')
>>> number_list
['1', '2', '3', '4', '5']
>>>
```

# Splitting a String

```
def main():
    # Create a string with multiple words.
    my_string = 'One two three four'

    # Split the string.
    word_list = my_string.split()

    # Print the list of words.
    print(word_list)

# Call the main function.
main()
```

```
def main():
    # Create a string with a date.
    date_string = '11/26/2018'

    # Split the date.
    date_list = date_string.split('/')

    # Display each piece of the date.
    print('Month:', date_list[0])
    print('Day:', date_list[1])
    print('Year:', date_list[2])

# Call the main function.
main()
```

# Summary

- This chapter covered:
  - String operations, including:
    - Methods for iterating over strings
    - Repetition and concatenation operators
    - Strings as immutable objects
    - Slicing strings and testing strings
    - String methods
    - Splitting a string